

SLOT 7

CHAPTER 06: HANDLING NAVIGATION WITH ROUTES

React Router & SPA Navigation – Routes, Links, Parameters, Query String, Redirect, Nested Routes & 404

Lớp: SE1910 | Định hướng: Project-Based Learning | Đối tượng: Bachelor

MỤC TIÊU TRUNG TÂM: Chuyển tư duy từ “đổi màn hình bằng state thủ công” sang “URL là một phần trạng thái điều hướng của ứng dụng”, từ đó xây dựng SPA có route rõ ràng, chia sẻ được URL, hỗ trợ back/forward và mở đường cho Lab 02.

Thông tin nhanh

Thuộc tính	Nội dung
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Lớp	SE1910
Slot	Slot 7
Chủ đề	Chapter 06 – Handling Navigation with Routes
Công nghệ	ReactJS, JavaScript/JSX, Node 21, Vite/CRA, React Router DOM
Nền tảng trước đó	Slot 6 – React Hook + Lab 01
Đích tiếp theo	Client–Server Communication, Fetching/Caching Data, Lab 02 – SPA

Cách sử dụng giáo trình

- Giảng viên: dùng các bảng mental model, checkpoint và mini-lab để dẫn dắt trên lớp.
- Sinh viên: đọc theo thứ tự, chạy từng ví dụ và chỉ chuyển phần khi checkpoint đạt.
- Mỗi nội dung kỹ thuật đều đi theo: Mục tiêu → Khái niệm → Ví dụ → Giải thích → Lỗi → Best practice → Bài tập.
- Không dùng AI để sinh toàn bộ lời giải trước khi tự thiết kế route table và trace URL.

MỤC LỤC NỘI DUNG

- 0. Sinh viên cần chuẩn bị gì trước khi học Slot 7
- I. Tóm tắt các chủ đề & mục tiêu trong bài học
- II. Trọng tâm cốt lõi & kỹ năng sinh viên cần nắm vững
- III. Ứng dụng thực tiễn kiến thức của bài học trong sản phẩm thực tế
- IV. Nội dung chi tiết về hướng dẫn việc thực hiện và triển khai
- V. Kinh nghiệm thực tiễn, Tips
- VI. Các bài tập chủ đề và bài tập tổng hợp hệ thống
- VII. Từ điển thuật ngữ – Foundation Glossary
- VIII. Research Base và AI-Based Learning
- IX. Sinh viên cần tự kiểm tra gì sau buổi học

MENTAL MODEL XUYỀN SUỐT: URL → Router → Route match → View. Với dữ liệu trên URL: path params dùng để định danh tài nguyên; query params dùng cho filter/search/page; navigation logic dùng useNavigate/Navigate khi có lý do rõ ràng.

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT 7

0.1 Kiến thức cần ôn từ Slot 6

Kiến thức	Cần nhớ	Liên hệ Slot 7
useState	Quản lý dữ liệu thay đổi và làm UI re-render	Không dùng state như bộ router thủ công khi URL cần phản ánh view
useEffect	Đồng bộ với side effect/external system	Có thể dùng cho title hoặc theo dõi location, nhưng routing không phụ thuộc useEffect
useContext	Chia sẻ dữ liệu xuyên cây component	Có thể kết hợp với auth/theme, không thay thế Router
useRef	DOM/value không gây re-render	Không phải công cụ điều hướng
Custom Hook	Tái sử dụng logic hook	Sau này có thể bọc logic navigation/domain khi có bằng chứng cần tái sử dụng
Rules of Hooks	Hook gọi top-level trong function component/custom hook	useNavigate/useParams/useSearchParams cũng phải tuân theo Rules of Hooks
Form/State	Controlled input, immutable update	Form có thể điều hướng sau submit bằng useNavigate

0.2 Kiến thức nền từ các slot trước

- Slot 2: JSX, component, rendering.
- Slot 3: Function component, component composition.
- Slot 4: React Bootstrap, layout và UI component.
- Slot 5: Props, state, context, one-way data flow.
- Slot 6: Hooks, form input và state management cơ bản.

0.3 Kỹ năng tối thiểu trước giờ học

- Tạo/chạy project React bằng Vite hoặc Create React App.
- Tạo function component và import/export đúng.
- Dùng useState/useEffect cơ bản.
- Dùng map() để render collection.
- Xử lý click và controlled input.

- Đọc lỗi terminal và Browser Console.

0.4 Công cụ & thư viện

Công cụ	Mục đích	Checkpoint trước giờ học
Node 21	npm/npmx và runtime toolchain	node -v
npm/npmx	Cài package và scaffold	npm -v, npx --version
IntelliJ IDEA / VS Code	IDE	Mở được project, terminal hoạt động
Chrome/Edge	Run & DevTools	Mở Console/Network
React DevTools	Quan sát component tree	Cài nếu có thể
Git/GitHub	Checkpoint/source	git --version
react-router-dom	Routing library	Cài trong buổi học

0.5 Bài chuẩn bị trước buổi học

1. Tạo app có Header, Footer và Main Content.
2. Tạo ba component tĩnh Home, About, Contact.
3. Tạo danh sách sản phẩm giả lập, hiển thị bằng map().
4. Đọc trước khái niệm Route, Link, NavLink.
5. Viết 3 câu trả lời: Vì sao SPA không nên reload toàn bộ trang? Vì sao URL nên phản ánh view? Vì sao state currentPage khó mở rộng?

0.6 Pre-check không dùng AI

Câu hỏi	Tự trả lời
SPA là gì?	
Nếu dùng <a href> để điều hướng nội bộ, chuyện gì có thể xảy ra?	
State currentPage có thể đổi view không? Hạn chế?	
URL có ích gì cho bookmark/back-forward/share?	
Route params và query params có thể khác nhau ở điểm nào?	

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1. Bảng tóm tắt kiến thức Slot 6

Nhóm kiến thức	Nội dung chính	Ý nghĩa	Ví dụ	Vấn đề Slot 7 giải quyết
useState	State trong function component	Điều khiển UI theo dữ liệu	searchText, selectedProduct	Không thích hợp để tự mô phỏng navigation quy mô lớn
useEffect	Side effect / synchronization	Timer, title, API awareness	document.title	Routing cần mô hình URL chuyên biệt, không phải effect thủ công
useContext	Chia sẻ dữ liệu	Tránh prop drilling hợp lý	ThemeContext	Context không cung cấp URL/history/deep link
useRef	DOM/value không re-render	Focus/input reference	inputRef	Không thay thế navigation state
Custom Hook	Tái sử dụng logic	Tách logic khỏi UI	useLocalStorage	Navigation hook của Router cung cấp API phù hợp
Rules of Hooks	Ổn định thứ tự hook	Tránh lỗi runtime/logic	Hook ở top level	useNavigate/useParams cũng tuân thủ
Form handling	Controlled form	Quản lý dữ liệu nhập	ProductForm	Sau submit có thể cần chuyển route
UI state	Re-render theo state	Tương tác trong một view	Modal/filter	Ứng dụng nhiều view cần route architecture

2. Định vị Slot 7 trong chuỗi học tập

Slot	Chủ đề	Mối liên hệ với Slot 7	Slot 7 hỗ trợ
6	React Hook + Lab 01	Cung cấp state/effect/context/ref	Làm component đủ khả năng tương tác trước khi có routing
7	Handling Navigation with Routes	Trọng tâm hiện tại	Xây client-side routing, URL state, dynamic route
8	Client–Server Communication	View nào sẽ gọi API	Route detail/list là điểm gắn request phù hợp
9	Fetching & Caching Data	Data theo route/query	Tách navigation khỏi data fetching
10	React Router and SPA	Đào sâu SPA/router	Nền tảng trực tiếp
11	Lab 02 + Assignment 01 + Project Assistant 01	Áp dụng route vào sản phẩm	Route table, dynamic detail, 404, layout

3. Bối cảnh bài học: từ một view đến nhiều view

Ở các slot trước, sinh viên đã có thể tạo component, quản lý state, chia sẻ Context và xây UI tương tác. Tuy nhiên, nếu ứng dụng có Home, Products, Product Detail, Contact, Login và Dashboard mà chỉ dùng biến currentPage để chuyển component, URL không phản ánh đúng view, người dùng khó bookmark/chia sẻ màn hình, và browser Back/Forward không còn là một phần tự nhiên của kiến trúc.

VẤN ĐỀ: Khi số view tăng, navigation không còn là một biến state cục bộ; nó trở thành một phần kiến trúc ứng dụng gắn với URL và browser history.

```
// Cách thủ công - chỉ phù hợp demo rất nhỏ
const [currentPage, setCurrentPage] = useState("home");
```

```

return currentPage === "home"
  ? <Home />
  : currentPage === "about"
    ? <About />
    : <Contact />;

```

Cách trên có thể chạy, nhưng URL vẫn không mô tả view. React Router giải quyết bài toán bằng cách ánh xạ URL với component và đồng bộ navigation với history của trình duyệt.

4. Mục tiêu học tập

- Giải thích SPA navigation và phân biệt với navigation web truyền thống.
- Cài đặt và cấu hình react-router-dom.
- Dùng BrowserRouter, Routes, Route để tạo route tĩnh.
- Dùng Link/NavLink cho navigation nội bộ.
- Dùng useParams cho route động.
- Dùng useSearchParams cho filter/search/page trên URL.
- Dùng useNavigate cho điều hướng theo logic và Navigate cho redirect trong render.
- Tạo nested route + Outlet ở mức nền tảng.
- Tạo 404 route và xử lý resource-not-found hợp lý.
- Vận dụng vào mini SPA chuẩn bị Lab 02.

5. Khái niệm trung tâm

Khái niệm	Ý nghĩa ngắn
SPA	Một ứng dụng web phía client có thể hiển thị nhiều view mà không cần full reload cho mọi navigation nội bộ.
Client-side routing	Router trên client quyết định view theo URL.
Route	Mapping giữa path pattern và element/view.
BrowserRouter	Routing context gắn với browser URL/history.
Routes	Bộ matcher của các Route.
Link / NavLink	Navigation UI nội bộ; NavLink hỗ trợ active state.
useParams	Đọc path parameters như :id.
useSearchParams	Đọc/cập nhật query string.
useNavigate	Điều hướng bằng code theo sự kiện/logic.
Navigate	Component redirect trong render.
Outlet	Vị trí hiển thị child route trong layout cha.
404	Fallback view khi không route nào khớp.

6. Flow tổng quát của Slot 7

```

Slot 6: Hook / State / Form
    ↓
Ứng dụng có nhiều view
    ↓
State currentPage trở nên khó mở rộng
    ↓
SPA cần client-side routing
    ↓

```

```
BrowserRouter → Routes → Route
```

```
↓
```

```
Link / NavLink
```

```
↓
```

```
useParams / useSearchParams
```

```
↓
```

```
useNavigate / Navigate
```

```
↓
```

```
Nested Route / Outlet / 404
```

```
↓
```

```
Lab 02: Create Single Page Application
```

7. Mini Demo App – Product Navigation Demo

View	URL	Kiến thức
Home	/	Static route
Products	/products	List route + query params
Product Detail	/products/:id	Dynamic route + useParams
About	/about	Static route
Contact	/contact	Static route
Dashboard	/dashboard/*	Nested route + Outlet
NotFound	*	Fallback 404

8. Tư duy kiến trúc quan trọng

- URL là một phần trạng thái điều hướng của ứng dụng.
- Route chọn view; route không tự động fetch dữ liệu.
- Path param phù hợp định danh tài nguyên; query param phù hợp filter/search/page.
- Link/NavLink dành cho user-visible navigation; useNavigate dành cho logic/sự kiện có lý do.
- Nested route tái sử dụng layout thay vì copy layout.
- Navigation và data fetching là hai trách nhiệm khác nhau, sẽ kết nối ở các slot tiếp theo.

9. Kết luận phần tóm tắt

Slot 7 đánh dấu bước chuyển từ React UI tương tác trong một view sang React SPA có cấu trúc nhiều view dựa trên URL. Sinh viên không chỉ cần viết đúng cú pháp Router mà phải thiết kế URL rõ ràng, trace được route matching và giải thích vì sao mỗi dữ liệu nằm ở path, query string hay component state.

10. Tài liệu tham khảo theo prompt

- React Router documentation: reactrouter.com/en/main
- React Router Navigate documentation
- MDN – SPA Glossary
- MDN – Introduction to server-side programming
- Slide Ch06 1–22
- Lab 02 handout
- React.dev – component/hooks review khi cần

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. SPA Navigation là gì?

Mục tiêu

Hiểu navigation trong SPA là chuyển view phía client theo URL/history mà không cần full-document reload cho mỗi điều hướng nội bộ.

Khái niệm

SPA tải ứng dụng ban đầu, sau đó JavaScript/React thay đổi view. URL vẫn có thể thay đổi để mô tả vị trí logic của người dùng.

Ví dụ

Ví dụ luồng

```
User click Products
→ URL /products
→ Router match /products
→ ProductList render
→ Không full reload toàn bộ document
```

Giải thích

Điểm quan trọng: “single page” không có nghĩa là “single view”. Một SPA có thể có nhiều view được ánh xạ bởi nhiều URL.

Lỗi thường gặp

- Nghĩ SPA không cần URL.
- Dùng state currentPage cho mọi màn hình.

Best practices

- Thiết kế URL trước khi code route.
- Giữ URL có nghĩa và có thể chia sẻ.

Bài tập/câu hỏi kiểm tra

- Giải thích bằng lời sự khác nhau giữa một HTML document và nhiều React views.

2. Web navigation truyền thống vs SPA navigation

Tiêu chí	Web truyền thống	SPA navigation
Full reload	Thường có khi chuyển tải liệu HTML	Không cần cho điều hướng nội bộ thông thường
Rendering	Server có thể trả HTML mới	Client chọn/render React view
Back/Forward	Tự nhiên theo document history	Router đồng bộ với browser history
State	Có thể mất/khôi phục theo request	Client state có thể giữ nếu component/layout vẫn tồn tại
URL	Tài nguyên/server page	Client route/view và có thể đồng thời tương tác server
UX	Có thể gián đoạn hơn	Chuyển view mượt hơn
SEO/deployment	Server route trực tiếp	Cần hiểu fallback/deployment cho SPA

Không tuyệt đối hóa: SPA không “tốt hơn mọi trường hợp”, và web truyền thống không “lỗi thời”. Mục tiêu Slot 7 là hiểu mô hình client-side routing trong React.

3. Vì sao cần React Router?

Nếu chỉ dùng state để chọn màn hình, ứng dụng thiếu deep link, URL không thể chia sẻ đúng view, browser Back/Forward khó hoạt động tự nhiên, và route hierarchy khó tổ chức. React Router cung cấp cơ chế mapping URL → component view, navigation UI, dynamic parameter, nested route, redirect và fallback.

```
URL /products/42
  ↓
BrowserRouter
  ↓
Routes
  ↓
<Route path="/products/:id" ...>
  ↓
ProductDetail
  ↓
useParams() → id = "42"
```

4. BrowserRouter, Routes, Route

Mục tiêu

Tạo router context và ánh xạ các URL tĩnh với component.

Khái niệm

BrowserRouter kết nối router với address bar/history; Routes chọn route phù hợp; Route khai báo path và element.

Ví dụ

Ví dụ

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="/contact" element={<Contact />} />
  </Routes>
</BrowserRouter>
```

Giải thích

path là mẫu URL; element là React element được hiển thị khi route khớp. Router context phải bao quanh các component dùng Router APIs.

Lỗi thường gặp

- Quên BrowserRouter.
- Đặt Route ngoài Routes.
- Import sai component hoặc path typo.

Best practices

- Giữ route table nhỏ, rõ ràng ở giai đoạn học.
- Xác minh route bằng nhập URL trực tiếp trước khi thêm menu.

Bài tập/câu hỏi kiểm tra

- Tạo ba route /, /about, /contact và ghi URL → component tương ứng.

5. Link và NavLink

Mục tiêu

Tạo menu SPA không full reload và hiển thị active state.

Khái niệm

Link tạo navigation nội bộ; NavLink có thêm trạng thái active để style menu hiện tại.

Ví dụ

Ví dụ

```
import { NavLink } from "react-router-dom";

<NavLink
  to="/products"
  className={({ isActive }) => isActive ? "active-link" : "nav-link"}
>
  Products
</NavLink>
```

Giải thích

NavLink phù hợp cho menu vì nó cho biết route hiện hành có active hay không. Thẻ <a> vẫn phù hợp cho link ngoài hoặc trường hợp cổ ý browser navigation.

Lỗi thường gặp

- Dùng <a href> cho mọi navigation nội bộ.
- Link nằm ngoài Router context.
- Style active không xử lý hàm className.

Best practices

- Dùng NavLink cho navigation menu.
- Dùng Link cho CTA nội bộ đơn giản.
- Dùng <a> cho URL ngoài ứng dụng.

Bài tập/câu hỏi kiểm tra

- So sánh 3 trường hợp: menu nội bộ, link website ngoài, button submit rồi điều hướng. Chọn API phù hợp.

6. Route params và useParams

Mục tiêu

Tạo một detail route có thể phục vụ nhiều bản ghi.

Khái niệm

Route parameter nằm trong path pattern, ví dụ /products/:id. useParams đọc giá trị từ URL.

Ví dụ

Ví dụ

```
<Route path="/products/:id" element={<ProductDetail />} />

function ProductDetail() {
  const { id } = useParams();
  const product = products.find(p => String(p.id) === id);
  ...
}
```

Giải thích

URL `/products/101` khớp pattern `/products/:id` và cho `id = "101"`. Param URL thường là chuỗi, vì vậy cần đổi chiều kiểu với ID dữ liệu.

Lỗi thường gặp

- Tên param không khớp: `:productId` nhưng đọc `id`.
- So sánh `number === string`.
- Không xử lý `product undefined`.

Best practices

- Một route pattern cho nhiều resource IDs.
- Luôn có `resource-not-found` branch.

Bài tập/câu hỏi kiểm tra

- Thử `/products/1`, `/products/2`, `/products/999` và giải thích kết quả.

7. Query params và `useSearchParams`

Mục tiêu

Dùng URL để giữ trạng thái `filter/search/page` có thể chia sẻ.

Khái niệm

Query params xuất hiện sau dấu `?`. `useSearchParams` cho phép đọc và cập nhật query string.

Ví dụ

Ví dụ

```
import { useSearchParams } from "react-router-dom";

const [searchParams, setSearchParams] = useSearchParams();
const category = searchParams.get("category") || "all";

function changeCategory(value) {
  if (value === "all") setSearchParams({});
  else setSearchParams({ category: value });
}
```

Giải thích

Path param thường định danh resource chính; query param thường mô tả cách xem collection như `filter`, `search`, `sort` hoặc `page`.

Lỗi thường gặp

- Không xử lý `null`.
- UI filter thay đổi nhưng URL không cập nhật.
- Dùng query param cho primary ID khi path rõ ràng hơn.

Best practices

- Giữ query keys ngắn, rõ nghĩa.
- Khi URL là source of truth, UI phải đọc ngược từ URL.

Bài tập/câu hỏi kiểm tra

- Tạo `/products?category=phone` và `/products?category=laptop`; kiểm tra dropdown phản ánh URL.

8. `useNavigate`

Mục tiêu

Điều hướng bằng code sau một hành động hoặc logic.

Khái niệm

useNavigate trả về hàm navigate, phù hợp khi navigation là hậu quả của event/logic, ví dụ submit thành công.

Ví dụ

Ví dụ

```
const navigate = useNavigate();

function handleSubmit(event) {
  event.preventDefault();
  // validate / save frontend data
  navigate("/products");
}
```

Giải thích

Đừng dùng navigate chỉ vì có thể. Nếu navigation là một link người dùng nhìn thấy, Link/NavLink thường rõ nghĩa hơn.

Lỗi thường gặp

- Gọi navigate() trực tiếp trong render.
- Lạm dụng navigate cho menu.
- Điều hướng tới path không tồn tại.

Best practices

- Dùng trong handler hoặc effect có điều kiện rõ ràng.
- Có thể dùng navigate(-1) cho back khi UX phù hợp.

Bài tập/câu hỏi kiểm tra

- Viết form giả lập, sau submit chuyển về /products; giải thích vì sao useNavigate hợp lý.

9. Navigate và redirect

Mục tiêu

Hiểu redirect trong JSX/route definition.

Khái niệm

Navigate là component khiến route render chuyển hướng đến URL khác; thường dùng cho alias route hoặc conditional redirect.

Ví dụ

Ví dụ

```
<Route
  path="/old-products"
  element={<Navigate to="/products" replace />}
/>
```

Giải thích

replace giúp thay entry hiện tại trong history trong các trường hợp không muốn người dùng Back quay lại URL cũ.

Lỗi thường gặp

- Redirect vòng lặp.
- Redirect không có điều kiện rõ ràng.
- Không hiểu replace ảnh hưởng history.

Best practices

- Dùng cho URL legacy/alias hoặc conditional view redirect.
- Giữ redirect đơn giản ở giai đoạn này.

Bài tập/câu hỏi kiểm tra

- Tạo /old-home → / và kiểm tra Browser Back.

10. Nested routes và Outlet**Mục tiêu**

Tái sử dụng layout cha cho nhiều route con.

Khái niệm

Nested route mô hình hóa hierarchy URL/UI. Outlet là vị trí layout cha dành cho child route đang khớp.

Ví dụ

Ví dụ

```
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} />
  <Route path="profile" element={<Profile />} />
  <Route path="orders" element={<Orders />} />
</Route>

// DashboardLayout.jsx
return (
  <div>
    <DashboardMenu />
    <Outlet />
  </div>
);
```

Giải thích

Các path con tương đối với route cha. Index route đại diện child mặc định của /dashboard.

Lỗi thường gặp

- Quên Outlet.
- Viết child path tuyệt đối không cần thiết.
- Không hiểu index route.

Best practices

- Vẽ route tree trước.
- Chỉ dùng nested route khi UI thực sự chia sẻ layout.

Bài tập/câu hỏi kiểm tra

- Tạo dashboard có /dashboard, /dashboard/profile, /dashboard/orders.

11. Trang 404 và hai loại “Not Found”

Client 404 xảy ra khi không route pattern nào khớp. Resource not found xảy ra khi route khớp nhưng ID không tồn tại trong dữ liệu. Hai trường hợp phải được phân biệt vì sau này backend cũng có thể trả HTTP 404 cho resource.

```
<Route path="*" element={<NotFound />} />

// Trong ProductDetail
if (!product) {
  return <p>Product not found.</p>;
}
```

Tình huống	Ví dụ	Kết quả đúng
Route không tồn tại	/abc	NotFound page

Tình huống	Ví dụ	Kết quả đúng
Route tồn tại, resource không có	/products/999	Product not found
Route hợp lệ, resource có	/products/2	Product detail

12. useLocation – awareness hữu ích

useLocation trả về location hiện tại và giúp component biết pathname/search/state. Prompt đặt useLocation trong nhóm khái niệm trung tâm; ở Slot 7 chỉ cần nhận diện và dùng cho quan sát/debug hoặc cập nhật title đơn giản, không biến nó thành state store.

```
const location = useLocation();
console.log(location.pathname);
console.log(location.search);
```

13. Kỹ năng sinh viên cần đạt

- Cài react-router-dom và xác minh package.
- Bọc app bằng BrowserRouter.
- Tạo static/dynamic routes.
- Tạo menu Link/NavLink và active state.
- Đọc path param bằng useParams.
- Đọc/cập nhật query string bằng useSearchParams.
- Điều hướng bằng useNavigate đúng ngữ cảnh.
- Redirect bằng Navigate.
- Tạo nested route và Outlet.
- Xử lý client 404 và resource not found.
- Trace URL → match → component → param/query → UI.

14. Những hiểu sai phổ biến

Hiểu sai	Sửa lại mental model
SPA không cần URL	SPA vẫn có thể và nên có URL có nghĩa cho view.
Menu = Router	Menu chỉ là UI; Router mới match URL.
Dùng state là đủ cho mọi navigation	State không thay thế history/deep link/shareable URL.
Route params = query params	Path params định danh cấu trúc/resource; query thường mô tả view/filter.
useNavigate dùng cho mọi link	Link/NavLink rõ nghĩa hơn cho navigation user-visible.
Nested route = component lồng nhau	Nested route là hierarchy route + UI layout; Outlet là render point.
React Router tự gọi backend	Router chọn view, không tự fetch dữ liệu.
404 route = missing resource	Route 404 khác resource not found.

15. Bài học quan trọng

4 CÂU HỎI THIẾT KẾ ROUTE: View nào cần URL riêng? Dữ liệu nào là path param? Dữ liệu nào là query param? Navigation này nên là Link/NavLink hay do logic dùng navigate?

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

1. Routing trong các loại sản phẩm

Loại sản phẩm	Route điển hình	Ý nghĩa
E-commerce	/products, /products/:id, /cart, /checkout	List → detail → transaction flow
Quản lý	/dashboard, /products, /orders, /users	Module-based navigation
Học tập	/courses, /courses/:id, /progress	Deep link vào course/lesson
Mạng xã hội	/feed, /profile/:id, /messages, /settings	Resource + account navigation
News	/news, /news/:id, /news?category=tech	Detail + query filter

2. Tình huống thực tế và API Router phù hợp

Tình huống	Công cụ	Lý do
User bấm Products	NavLink/Link	Navigation trực tiếp từ UI
User chọn product 42	Link `/products/42`	Deep link resource
Filter category=phone	useSearchParams	Filter nên phản ánh trên URL
Submit form xong về list	useNavigate	Navigation là hậu quả của logic
URL cũ đổi sang URL mới	Navigate	Redirect declarative
Dashboard có tab con	Nested Route + Outlet	Tái sử dụng layout
Route không tồn tại	path=""	Client fallback

3. Case study: Product Navigation Demo App

```

ROUTE MAP
/
├─ /products
│   └─ /products/:id
├─ /about
├─ /contact
├─ /dashboard
│   └─ /dashboard/profile
│       └─ /dashboard/orders
└─ * → NotFound
  
```

Products có thể dùng query string như `/products?category=phone&sort=asc`. Đây là ví dụ để phân biệt route hierarchy với trạng thái view/filter.

4. Lợi ích thực tiễn

- Deep linking: gửi link thẳng tới view/resource.
- Back/Forward hoạt động nhất quán với navigation.
- Bookmarkable URL: người dùng lưu vị trí.
- Route table làm kiến trúc UI dễ đọc hơn.
- Dynamic route giúp một component detail xử lý nhiều bản ghi.
- Nested route giảm copy layout.

5. Liên hệ với Spring Boot ở các slot sau

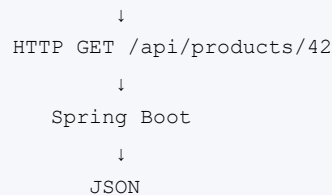
PHÂN BIỆT BẮT BUỘC: React route `/products/42` quyết định view phía client. REST endpoint `GET /api/products/42` sau này cung cấp dữ liệu từ server. Chúng có thể trông giống URL, nhưng trách nhiệm hoàn toàn khác.

Navigation flow hôm nay:

Browser URL `/products/42` → React Router → ProductDetail → local data

Full-stack flow sau này:

Browser URL `/products/42` → React Router → ProductDetail



IV. NỘI DUNG CHI TIẾT VỀ HƯỚNG DẪN VIỆC THỰC HIỆN VÀ TRIỂN KHAI

QUY TẮC THỰC HÀNH: Mỗi phần: thiết kế → viết thay đổi nhỏ → chạy → kiểm tra URL và view → xem Console → chỉ tiếp tục khi checkpoint PASS.

PHẦN A. SPA Navigation và vấn đề cần giải quyết

A1. Cách thủ công bằng state

```
function App() {
  const [page, setPage] = useState("home");

  return (
    <>
      <button onClick={() => setPage("home")}>Home</button>
      <button onClick={() => setPage("about")}>About</button>
      {page === "home" && <Home />}
      {page === "about" && <About />}
    </>
  );
}
```

Cách này minh họa state-driven UI nhưng không cung cấp URL riêng cho view. Hãy yêu cầu sinh viên thử Back/Forward và copy URL để thấy hạn chế.

A2. Khi nào state vẫn phù hợp?

State vẫn phù hợp cho tab cục bộ, modal, accordion hoặc interaction không cần URL riêng. Router được dùng khi navigation mang ý nghĩa application-level và người dùng hưởng lợi từ URL/history/deep link.

Checkpoint A

- Sinh viên giải thích được ít nhất 3 hạn chế của currentPage state đối với multi-view app.
- Sinh viên phân biệt được local UI state và navigation state trên URL.

PHẦN B. Cài đặt React Router DOM

```
npm install react-router-dom
npm run dev
```

Sau cài đặt, kiểm tra `package.json` có dependency và app vẫn compile. Không nâng/hạ phiên bản tùy tiện giữa buổi học nếu project course baseline đã khóa phiên bản.

Checkpoint B

- Terminal không có blocking error.
- Import được BrowserRouter, Routes, Route.

PHẦN C. Cấu hình BrowserRouter và route cơ bản

C1. main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

C2. App.jsx

```
import { Routes, Route } from "react-router-dom";
import Home from "./pages/Home.jsx";
import About from "./pages/About.jsx";
import Contact from "./pages/Contact.jsx";

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  );
}
```

C3. Verify bằng URL trực tiếp

URL	Expected	Actual/PASS
/	Home	
/about	About	
/contact	Contact	

Chỉ khi route trực tiếp hoạt động mới thêm Link. Điều này giúp tách lỗi route config khỏi lỗi menu.

PHẦN D. Layout, Link và NavLink

D1. Layout component

```
import { NavLink, Outlet } from "react-router-dom";
```

```
export default function Layout() {
  const navClass = ({ isActive }) => isActive ? "nav-link active" : "nav-link";

  return (
    <>
      <header>
        <nav>
          <NavLink to="/" end className={navClass}>Home</NavLink>
          <NavLink to="/products" className={navClass}>Products</NavLink>
          <NavLink to="/about" className={navClass}>About</NavLink>
          <NavLink to="/contact" className={navClass}>Contact</NavLink>
        </nav>
      </header>
      <main><Outlet /></main>
      <footer>SBA301 • Slot 7</footer>
    </>
  );
}
```

D2. App routes dùng layout

```
<Routes>
  <Route element={<Layout />}>
    <Route path="/" element={<Home />} />
    <Route path="/products" element={<ProductList />} />
    <Route path="/products/:id" element={<ProductDetail />} />
    <Route path="/about" element={<About />} />
    <Route path="/contact" element={<Contact />} />
  </Route>
</Routes>
```

Layout route tái sử dụng Header/Footer; Outlet là nơi page component hiện ra. `end` trên Home giúp tránh Home bị active ở mọi URL bắt đầu bằng `/`.

Checkpoint D

- Click menu làm URL và view đổi.
- Không có full-page navigation ngoài ý muốn.
- Active menu đúng route.
- Header/Footer giữ nguyên khi đổi view.

PHẦN E. Route params và useParams

E1. Mock data

```
export const products = [
  { id: 1, name: "Laptop Pro", category: "laptop", price: 1500 },
  { id: 2, name: "Phone X", category: "phone", price: 900 },
  { id: 3, name: "Tablet Air", category: "tablet", price: 700 },
  { id: 4, name: "Phone Mini", category: "phone", price: 550 }
];
```

E2. Product list

```
import { Link } from "react-router-dom";
import { products } from "../data/products.js";

export default function ProductList() {
  return (
    <section>
```

```

<h1>Products</h1>
{products.map(product => (
  <article key={product.id}>
    <h2>{product.name}</h2>
    <p>{product.category} • ${product.price}</p>
    <Link to={`/${products}/${product.id}`}>View detail</Link>
  </article>
))}
</section>
);
}

```

E3. Product detail

```

import { Link, useParams } from "react-router-dom";
import { products } from "../data/products.js";

export default function ProductDetail() {
  const { id } = useParams();
  const product = products.find(item => String(item.id) === id);

  if (!product) {
    return (
      <section>
        <h1>Product not found</h1>
        <Link to="/products">Back to products</Link>
      </section>
    );
  }

  return (
    <section>
      <h1>{product.name}</h1>
      <p>Category: {product.category}</p>
      <p>Price: ${product.price}</p>
      <Link to="/products">Back to products</Link>
    </section>
  );
}

```

Checkpoint E

Test	Expected
/products/1	Laptop Pro
/products/2	Phone X
/products/999	Product not found

PHẦN F. Query params và useSearchParams

```

import { useSearchParams } from "react-router-dom";
import { products } from "../data/products.js";

export default function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();

```

```

const category = searchParams.get("category") || "all";
const keyword = searchParams.get("q") || "";

const visibleProducts = products.filter(product => {
  const matchCategory = category === "all" || product.category === category;
  const matchKeyword = product.name.toLowerCase().includes(keyword.toLowerCase());
  return matchCategory && matchKeyword;
});

function updateCategory(value) {
  const next = new URLSearchParams(searchParams);
  if (value === "all") next.delete("category");
  else next.set("category", value);
  setSearchParams(next);
}

function updateKeyword(value) {
  const next = new URLSearchParams(searchParams);
  if (!value.trim()) next.delete("q");
  else next.set("q", value);
  setSearchParams(next);
}

return (
  <section>
    <h1>Products</h1>
    <input
      value={keyword}
      onChange={e => updateKeyword(e.target.value)}
      placeholder="Search products"
    />
    <select value={category} onChange={e => updateCategory(e.target.value)}>
      <option value="all">All</option>
      <option value="phone">Phone</option>
      <option value="laptop">Laptop</option>
      <option value="tablet">Tablet</option>
    </select>
    <p>{visibleProducts.length} product(s)</p>
    { /* render visibleProducts */ }
  </section>
);
}

```

Điểm học tập: filter state được mã hóa vào URL. Reload hoặc copy URL vẫn giữ ý nghĩa filter, miễn dữ liệu nguồn còn như cũ.

Checkpoint F

- Chọn Phone → URL có category=phone và list chỉ còn phone.
- Nhập keyword → q xuất hiện trên URL.
- Copy URL sang tab mới → filter đọc lại từ URL.

PHẦN G. useNavigate – điều hướng bằng logic

```
import { useNavigate } from "react-router-dom";
```

```
export default function AddProductDemo() {
  const navigate = useNavigate();

  function handleSubmit(event) {
    event.preventDefault();
    // demo: validation/save would happen here
    navigate("/products");
  }

  return (
    <form onSubmit={handleSubmit}>
      <input required placeholder="Product name" />
      <button type="submit">Save</button>
      <button type="button" onClick={() => navigate(-1)}>Cancel</button>
    </form>
  );
}
```

Navigation này xảy ra sau submit/cancel logic, vì vậy useNavigate phù hợp hơn Link trong handler.

PHẦN H. Navigate – redirect

```
import { Navigate } from "react-router-dom";

<Route path="/old-home" element={<Navigate to="/" replace />} />
<Route path="/old-products" element={<Navigate to="/products" replace />} />
```

Checkpoint H

- Nhập /old-products → URL thành /products.
- Quan sát Back để hiểu replace.

PHẦN I. Nested route và Outlet

```
// App.jsx
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} />
  <Route path="profile" element={<Profile />} />
  <Route path="orders" element={<Orders />} />
</Route>

// DashboardLayout.jsx
import { NavLink, Outlet } from "react-router-dom";

export default function DashboardLayout() {
  return (
    <section>
      <h1>Dashboard</h1>
      <nav>
        <NavLink to=".">Overview</NavLink>{" | "}
        <NavLink to="profile">Profile</NavLink>{" | "}
        <NavLink to="orders">Orders</NavLink>
      </nav>
      <Outlet />
    </section>
  );
}
```

Checkpoint I

URL	View trong Outlet
/dashboard	DashboardHome
/dashboard/profile	Profile
/dashboard/orders	Orders

PHẦN J. Trang 404

```
function NotFound() {
  return (
    <section>
      <h1>404 - Page not found</h1>
      <Link to="/">Return Home</Link>
    </section>
  );
}

// cuối route table
<Route path="*" element={<NotFound />} />
```

Trong React Router hiện đại, route matching có ranking; mục tiêu sơ phạm ở đây là đảm bảo wildcard fallback tồn tại và dễ nhìn trong route table.

PHẦN K. useLocation – mini awareness demo

```
import { useLocation } from "react-router-dom";

function RouteDebug() {
  const location = useLocation();
  return <small>{location.pathname}{location.search}</small>;
}
```

Dùng để quan sát location hoặc hỗ trợ logic phụ; không cần đưa vào core lab nếu thời gian hạn chế.

PHẦN L. Chuẩn bị Lab 02 – Create Single Page Application with ReactJS

Yêu cầu Lab 02 gợi ý	Bảng chứng
Layout SPA	Header/Footer/Main dùng Layout + Outlet
Ít nhất 5 route	Route table
Menu điều hướng	NavLink active
Dynamic route	/products/:id
Query/filter	/products?category=...
Redirect	/old-products → /products
404	path="*"
Không vi phạm Rules of Hooks	Code review + console sạch

PHẦN M. Debugging workflow cho routing

```
Navigation lỗi
↓
```

```

App compile chưa?
  ↓
BrowserRouter có bao quanh component không?
  ↓
URL có thay đổi đúng không?
  ↓
Routes có pattern phù hợp không?
  ↓
Component có render không?
  ↓
Nếu dynamic: param name/value/type đúng?
  ↓
Nếu query: search param key/value đúng?
  ↓
Nếu nested: parent có Outlet không?
  ↓
Sau cùng mới kiểm tra styling/detail UI

```

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. Bảng lỗi thường gặp và cách xử lý

Dấu hiệu	Nguyên nhân thường gặp	Cách chẩn đoán	Cách xử lý
useNavigate/useParams báo lỗi context	Component ngoài Router	Kiểm tra component tree	Đặt BrowserRouter ở entry point phù hợp
Click menu full reload	Dùng <a href> nội bộ	Xem Network/document reload	Đổi sang Link/NavLink nếu là internal SPA navigation
URL đúng nhưng blank	Route element/import lỗi	Console + route table	Kiểm tra import và element
Detail undefined	Param mismatch/type mismatch	Log params và data IDs	Đồng bộ param name, chuyển kiểu có chủ ý
Filter mất khi reload	Filter chỉ lưu local state	Xem URL có query không	Dùng useSearchParams khi filter nên nằm trên URL
Nested page không xuất hiện	Quên Outlet	Kiểm tra parent layout	Thêm <Outlet />
Redirect loop	Target quay về route redirect	Trace URL sequence	Sửa condition/target
404 không hiện	Không có wildcard hoặc route config lỗi	Nhập URL lạ trực tiếp	Thêm path="*" và kiểm tra config

2. Best practices tổng quát

- Vẽ route table trước khi code.
- Dùng danh từ số nhiều cho collection route nhất quán: /products, /courses, /rooms.
- Dynamic ID ở path: /products/:id.
- Filter/search/page ưu tiên query string khi cần chia sẻ URL.
- Dùng NavLink cho menu có active state.
- Dùng Link cho link nội bộ rõ nghĩa.
- Dùng useNavigate khi navigation là hậu quả của logic.
- Tách route-not-found và resource-not-found.
- Không nhét data fetching vào khái niệm Router; giữ separation of concerns.
- Giữ route scope vừa đủ cho sinh viên; protected routes/loaders/lazy routing học sau.

3. 5 câu hỏi trước khi thêm một route

1. View này có cần URL độc lập để bookmark/share/back-forward không?
2. URL nên là static, dynamic param hay query-driven?
3. Ai tạo navigation tới route này: menu, item detail, hay logic sau action?
4. Route này có dùng chung layout với nhóm route nào?
5. Nếu URL/resource sai thì người dùng thấy gì?

4. Accessibility awareness

Navigation phải dùng phần tử semantic phù hợp. Link nên có text mô tả; active state không chỉ dựa vào màu nếu triển khai production. Slot này chỉ giới thiệu awareness, chưa đi sâu accessibility audit.

VI. CÁC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG, Củng cố KIẾN THỨC

A. 10 bài tập chủ đề

Bài 1 – Static Routes

Tạo /, /about, /contact; mỗi route có component riêng; kiểm tra bằng URL trực tiếp.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 2 – Link

Header có Home/About/Contact; click không full reload.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 3 – NavLink

Menu active có class riêng; giải thích isActive.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 4 – Dynamic Route

Tạo /products/:id và hiển thị id bằng useParams.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 5 – Query Params

/products?category=phone; đọc category bằng useSearchParams.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 6 – useNavigate

Form giả lập submit xong navigate /products; nút Cancel navigate(-1).

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 7 – Navigate

/old-home → /; /old-products → /products.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 8 – Nested Route

/dashboard, /dashboard/profile, /dashboard/orders; dùng Outlet.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 9 – 404

URL không tồn tại hiển thị NotFound và link về Home.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

Bài 10 – Params + Link/Navigate

Product list có View Detail đi tới /products/:id; test ID hợp lệ và không tồn tại.

Bằng chứng: screenshot URL + view, route config, và 2–3 câu giải thích mental model.

B. Bài tập tổng hợp – SPA Navigation Demo Application

Mục tiêu: xây dựng SPA frontend có route architecture đủ để chuẩn bị Lab 02, nhưng chưa gọi backend.

Yêu cầu chức năng

- Layout gồm Header, Footer, Main Content.

- Menu: Home, Products, About, Contact.
- Products render mock data.
- Product Detail theo /products/:id.
- Filter Products bằng query parameter category.
- Button/link đi từ item đến detail.
- Ít nhất một redirect route.
- Nested Dashboard với tối thiểu 2 child routes.
- NotFound route.
- Có thể dùng useEffect từ Slot 6 để cập nhật document.title theo location ở phần mở rộng.

Yêu cầu kỹ thuật

- Function components.
- BrowserRouter, Routes, Route.
- Link hoặc NavLink.
- useParams.
- useSearchParams hoặc useNavigate.
- Navigate redirect.
- Outlet cho nested route.
- Không vi phạm Rules of Hooks.
- Không backend/REST bắt buộc ở bài này.

Cấu trúc thư mục đề xuất

```
src/
├── components/
│   ├── Header.jsx
│   ├── Footer.jsx
│   ├── ProductItem.jsx
│   └── Layout.jsx
├── pages/
│   ├── Home.jsx
│   ├── About.jsx
│   ├── Contact.jsx
│   ├── ProductList.jsx
│   ├── ProductDetail.jsx
│   ├── DashboardLayout.jsx
│   ├── DashboardHome.jsx
│   ├── Profile.jsx
│   ├── Orders.jsx
│   └── NotFound.jsx
├── data/
│   └── products.js
├── App.jsx
└── main.jsx
```

Acceptance checklist

#	Kiểm tra	PASS/FAIL
1	App chạy không blocking error	
2	Menu đổi URL/view không full reload	
3	/products hiển thị list	
4	/products/:id hiển thị đúng detail	

#	Kiểm tra	PASS/FAIL
5	Invalid product ID không crash	
6	Query filter hoạt động và URL cập nhật	
7	Redirect hoạt động	
8	Nested Dashboard + Outlet hoạt động	
9	Unknown route hiển thị 404	
10	Back/Forward hoạt động	

C. Bug Injection Lab

Bug cố ý	Triệu chứng	Sinh viên cần tìm
Route `:productId`, code đọc `id`	Detail undefined	Param naming mismatch
ID data number, param string	find() không thấy	Type mismatch
Xóa <Outlet />	Nested URL đổi nhưng child không hiện	Missing render point
Dùng href nội bộ	Có document navigation/reload	Wrong navigation element
Redirect /old → /old	Loop	Redirect target logic
Sai query key category/categoryId	Filter không phản ánh URL	Query naming mismatch

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ trong bài
Single Page Application	Ứng dụng web tải shell ban đầu và thay đổi view phía client cho navigation nội bộ.	Home → Products không full reload.
Client-side routing	Router phía browser quyết định view theo URL.	/products → ProductList.
Route	Khai báo ánh xạ path với element.	<Route path="/about" ...>.
Path	Mẫu URL của route.	/products/:id.
Routes	Container/matcher của Route.	<Routes>...</Routes>.
BrowserRouter	Router context dùng browser history/address bar.	Bao App ở main.jsx.
Link	Navigation nội bộ SPA.	<Link to="/products">.
NavLink	Link có active state.	Menu highlight route hiện tại.
useNavigate	Hook điều hướng bằng code.	navigate("/products").
useParams	Hook đọc path params.	id từ /products/:id.
useLocation	Hook đọc location hiện tại.	pathname/search.
useSearchParams	Hook đọc/cập nhật query params.	category=phone.
Navigate	Component redirect declarative.	/old-products → /products.
Nested Route	Route con nằm trong route cha/layout.	/dashboard/profile.
Outlet	Vị trí child route được render.	Trong DashboardLayout.
Index Route	Child mặc định của nested route.	/dashboard → DashboardHome.
Route Params	Giá trị động trong path.	/:id.
Query Params	Key/value sau dấu ?.	?category=phone.
Redirect	Chuyển URL theo rule/logic.	Navigate to=...
404 Page	Fallback khi route không khớp.	path="**".
Active Link	Navigation item ứng với route hiện tại.	NavLink isActive.
Navigation State	Trạng thái vị trí/điều hướng của ứng dụng.	pathname, search, history.
URL	Địa chỉ mô tả vị trí/tài nguyên/view.	/products/2?q=x.
History Navigation	Back/Forward stack của browser.	navigate(-1).
Layout Route	Route cung cấp layout dùng chung.	Header/Footer + Outlet.
Deep Link	URL truy cập trực tiếp một view cụ thể.	/products/42.
Resource Not Found	Route khớp nhưng dữ liệu không có.	/products/999.

VIII. RESEARCH BASE VÀ AI-BASED LEARNING

Mục tiêu của phần này là dùng AI như một công cụ nghiên cứu/kiểm chứng, không phải máy sinh lời giải thay sinh viên. Trước khi hỏi AI, sinh viên phải cung cấp route table, URL, expected result và actual result.

A. 10 prompt tự nghiên cứu chuyên sâu

1. Phân tích cơ chế client-side routing trong SPA và vai trò của History API. Dùng sơ đồ URL → history → router → view.
2. So sánh client-side routing với server-side routing về request, rendering, URL, history và deployment.
3. Giải thích cách React Router đồng bộ URL với component tree trong mô hình BrowserRouter + Routes + Route.
4. So sánh BrowserRouter, HashRouter và MemoryRouter; nêu use case và trade-off, không chỉ mô tả cú pháp.
5. Giải thích History API (pushState, replaceState, popstate) ở mức giúp sinh viên hiểu Back/Forward trong SPA.
6. Phân tích route params, query params và navigation state; đưa tiêu chí chọn và ví dụ sai/đúng.
7. Giải thích nested route và Outlet bằng route tree + UI tree; chỉ ra lỗi khi thiếu Outlet.
8. So sánh useNavigate và Navigate theo thời điểm sử dụng, data flow và khả năng gây redirect loop.
9. Phân tích cơ chế route matching/ranking trong React Router ở mức khái niệm; minh họa static vs dynamic vs wildcard.
10. Phân tích tác động của URL/navigation đến state management: dữ liệu nào nên nằm URL, local state, Context?

B. 10 prompt nghiên cứu thực tiễn production

1. Trong dự án React lớn, developer tổ chức route, layout và module boundaries như thế nào để dễ bảo trì?
2. Làm thế nào quản lý breadcrumb trong nested routes mà tránh hard-code ở nhiều component?
3. Protected route trong sản phẩm thật nên được thiết kế thế nào và vì sao redirect không thay thế authorization backend?
4. Làm thế nào giữ hoặc khôi phục scroll position khi back/forward trong SPA?
5. Route-level lazy loading giúp gì và có trade-off UX nào?
6. Query params trong filter/search/paging production nên encode thế nào để URL ổn định và shareable?
7. Thiết kế 404, resource-not-found và error boundary khác nhau thế nào?
8. Chiến lược test navigation bằng React Testing Library/Playwright ở mức integration là gì?
9. Làm thế nào tối ưu UX khi route transition có data loading?
10. Team lớn nên định nghĩa naming convention cho route paths, params và query keys như thế nào?

C. 10 prompt khám phá công nghệ và xu hướng

1. React Router v6/v7 thay đổi những khái niệm nào so với version cũ? Tách API ổn định với API mới.
2. So sánh React Router và TanStack Router theo routing model, type safety và data APIs.
3. Xu hướng tổ chức routing trong React application hiện đại là gì: centralized route config, file-based routing, framework routing?
4. Khi nào dùng framework routing như Next.js thay vì SPA routing thuần?
5. React Router loaders/actions là gì và thay đổi cách data fetching quanh route như thế nào?
6. Cộng đồng React hiện nay xử lý protected route và auth redirect như thế nào?
7. Xu hướng “URL as state” cho filter/search/pagination: lợi ích và giới hạn?
8. Best practices accessibility mới nhất cho navigation trong SPA gồm những gì?
9. Công cụ nào giúp debug React Router, history và navigation state?
10. Các lỗi hiệu năng thường gặp trong app có routing lớn và cách đo trước khi tối ưu?

D. Prompt AI dùng khi debug

Act as a React Router debugging tutor.

Do not rewrite my whole app.

Current URL: [URL]

Expected view: [EXPECTED]

Actual view: [ACTUAL]
 Routes config: [ROUTES]
 Relevant component: [CODE]
 Console error: [ERROR]

Trace in this order:

1. BrowserRouter context
2. URL
3. Route pattern
4. Route match
5. Params/search params
6. Component render
7. Resource lookup

Identify the FIRST broken stage and give one verification step.

E. AI-0 trước AI

Bảng chứng phải có trước khi hỏi AI	Giá trị
Current URL	
Expected component	
Actual component/result	
Route pattern	
Params/query values	
First console error	
Last code change	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

1. Kiểm tra kiến thức nền tảng

- SPA navigation là gì?
- SPA khác web truyền thống như thế nào?
- Vì sao cần React Router?
- BrowserRouter, Routes, Route có vai trò gì?
- Link khác <a> như thế nào?
- NavLink khác Link như thế nào?
- useParams dùng khi nào?
- useSearchParams dùng khi nào?
- useNavigate dùng khi nào?
- Navigate dùng khi nào?
- Nested route và Outlet dùng để làm gì?
- Client 404 khác resource-not-found thế nào?
- React Router có tự gọi Spring Boot không?

2. Kiểm tra kỹ năng thực hành

- Cài react-router-dom.
- Tạo static routes.
- Tạo Link/NavLink menu.
- Tạo dynamic route.
- Đọc params bằng useParams.
- Đọc/cập nhật query params.
- Điều hướng bằng useNavigate.
- Redirect bằng Navigate.
- Tạo nested route + Outlet.
- Tạo 404.
- Test Back/Forward và direct URL.
- Giải thích route table không dùng AI.

3. Kiểm tra sản phẩm thực hành

Tiêu chí	Đạt	Chưa đạt	Cần làm lại
Project chạy được			
Layout Header/Footer/Main			
Ít nhất 5 route			
Có dynamic route			
Menu navigation			
Query param/filter			
Nested route			
404 route			
Redirect route			
Back/Forward			
README/evidence nếu yêu cầu			

4. Mức độ tự đánh giá

Mức	Mô tả
Mức 1 – Chưa đạt	Chưa giải thích được SPA routing; chưa tạo được route cơ bản hoặc app không chạy.
Mức 2 – Đạt cơ bản	Hiểu route; tạo static/dynamic route và menu; còn lỗi nhỏ ở query/nested/404.
Mức 3 – Tốt	Giải thích được mental model; dùng Link/NavLink/useParams/useSearchParams/useNavigate/Navigate; nested route + 404 hoạt động; code rõ ràng.

5. Human Verification Gate – không dùng AI

Câu hỏi	Trả lời ngắn
1. URL /products/7 khớp route nào?	
2. useParams trả id ở dạng gì về mặt khái niệm?	
3. Vì sao filter category có thể nên dùng query param?	
4. Khi nào Link tốt hơn useNavigate?	
5. Khi nào Navigate phù hợp?	
6. Nếu /products/999 thì 404 route có nhất thiết chạy không? Vì sao?	
7. Nếu nested URL đúng nhưng child không hiện, kiểm tra gì đầu tiên?	
8. React route và REST endpoint khác nhau thế nào?	
9. Browser Back liên quan gì tới routing?	
10. Vì sao SPA vẫn cần URL có nghĩa?	

6. Việc cần làm nếu chưa đạt

- Làm lại static route /, /about, /contact.
- Làm lại NavLink active menu.
- Trace /products/:id bằng giấy trước rồi mới code.
- Test query params với category.
- Làm lại useNavigate sau submit.
- Tạo redirect alias và test Back.
- Tạo nested dashboard + Outlet.
- Tạo NotFound và phân biệt với Product not found.
- Nhờ giảng viên/AI giải thích dựa trên URL + route config + evidence, không yêu cầu viết lại toàn app.

7. Exit Ticket

Câu	Nội dung
Q1	Điền flow: URL → _____ → Route → Component.
Q2	/products/7: giá trị :id là gì?

Câu	Nội dung
Q3	Một ví dụ nên dùng query param thay vì path param.
Q4	Khác nhau giữa Link và useNavigate.
Q5	Vì sao React Router không tự fetch dữ liệu backend?

8. Reflection cuối buổi

```
Trước Slot 7, tôi nghĩ navigation là...
Bây giờ tôi hiểu URL là...
Route động của tôi là...
Query parameter của tôi là...
Một lỗi routing tôi gặp là...
Evidence giúp tôi tìm lỗi là...
Nếu kết nối Spring Boot, Router vẫn chịu trách nhiệm...
còn HTTP/API sẽ chịu trách nhiệm...
```

9. Cầu nối sang các slot tiếp theo

Slot 7:

URL → React Router → View → local/mock data

Slot 8+:

URL → React Router → View



KẾT LUẬN: Sinh viên hoàn thành Slot 7 khi không chỉ “click menu được”, mà có thể thiết kế route table, trace URL đến view, phân biệt path/query/navigation APIs và chứng minh routing độc lập với backend data fetching.

PHU LUC A. QUICK REFERENCE

Nhu cầu	API phù hợp	Ví dụ
Bọc app với browser history	BrowserRouter	<BrowserRouter><App/></BrowserRouter>
Khai báo route	Routes + Route	path="/about"
Menu có active	NavLink	to="/products"
Link nội bộ đơn giản	Link	View detail
Đọc ID path	useParams	/products/:id
Đọc/cập nhật filter URL	useSearchParams	?category=phone
Điều hướng sau logic	useNavigate	navigate("/products")
Redirect declarative	Navigate	to="/products" replace
Layout route	Outlet	Dashboard child content

Nhu cầu	API phù hợp	Ví dụ
Unknown route	*	NotFound

PHỤ LỤC B. ROUTE DESIGN WORKSHEET

View	URL	Static/Dynamic	Param	Query	Navigation source	Fallback
Home	/	Static	—	—	Menu	—
Collection						
Detail						
About						
Dashboard						
NotFound	*	Wildcard	—	—	Direct invalid URL	404

PHỤ LỤC C. GIT CHECKPOINT GỢI Ý

```
git status
git add .
git commit -m "Add SPA routing with React Router"
git log -1 --oneline
```

Chỉ commit sau khi route matrix và browser test đã PASS. Commit count không thay thế bằng chứng hiểu data/navigation flow.

PHỤ LỤC D. TÀI LIỆU NỘP / EVIDENCE PACKAGE

- Route table.
- Ảnh Home URL.
- Ảnh Products URL + query filter.
- Ảnh Product Detail URL.
- Ảnh nested Dashboard route.
- Ảnh NotFound route.
- Browser Console không có blocking error.
- Git log checkpoint.
- Đoạn giải thích 150–250 từ: URL → Route → View và React Router vs REST endpoint.